Dijkstra's or A*, searching helps determine the shortest or least costly route from one point to another, which is vital in robotics, autonomous vehicles, and game development. In decision-making systems, searching through a graph of possible moves or actions allows AI to assess potential outcomes and select the best course of action, a process that is fundamental in areas like machine learning, planning, and strategy games.

In this discussion, we will focus on two classical graph searching algorithms, also known as graph traversal algorithms: Breadth-First Search (BFS) and Depth-First Search (DFS). BFS explores a graph level by level, starting from a given vertex, called the root. It visits all neighbouring vertices at the current depth before moving on to those at the next depth level. Imagine you are at the entrance of a maze and want to explore it systematically. First, you check all the rooms (vertices) directly connected to the entrance. Then, you move to the rooms connected to those, continuing this process layer by layer.

On the other hand, DFS dives deep into the graph, exploring as far as possible along one path before backtracking. Think of DFS as selecting a corridor in the maze and following it until you reach a dead end. At that point, you backtrack to the last branching point and choose a different corridor to explore.

While BFS requires more memory to process since it keeps track of all the vertices at the current level, it guarantees finding the shortest path to the destination. DFS, in contrast, uses less memory because it only needs to store the current path, but it may not always find the shortest path to the destination.

For ease of understanding, we will first perform BFS and DFS on a tree. The program '*tree18.sagews*' shown below generates a tree called *Tree_18*.

```
T = Graph({0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [7, 8], 4: [9,
10], 5: [11, 12], 6: [13, 14]})
T.show( )
```

Notice that the first line of the above program is similar to the first line of the SageMath program '*wgraph17.sagews*', so no explanation is needed. Upon executing the program '*tree18.sagews*' using CoCalc, we obtain the tree *Tree_18*, as shown in Figure 4.

Now, let us perform BFS on the tree *Tree_18*. To do this, add the following lines of code at the end of the 'tree*18.sagews*' program. The BFS will begin from vertex 0.

```
bfs_order = list(T.breadth_first_search(0))
print("BFS Order:", bfs_order)
```

Upon executing the modified program, the following additional output will be displayed on the screen.

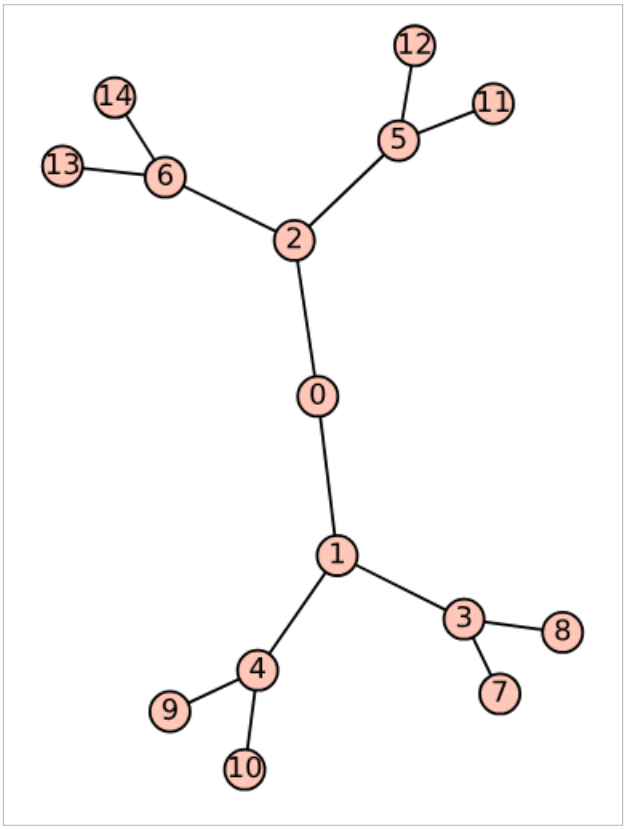

Figure 4: The tree *Tree_18*

```
BFS Order: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
```

Now, let us perform DFS on the tree *Tree_18*. To do this, add the following lines of code at the end of the '*tree18.sagews*' program. The DFS will begin from vertex 0.

```
dfs_order = list(T.depth_first_search(0))
print("DFS Order:", dfs_order)
```

Upon executing the modified program, the following additional output will be displayed on the screen.

```
DFS Order: [0, 2, 6, 14, 13, 5, 12, 11, 1, 4, 10, 9, 3, 8, 7]
```

A comparison of the BFS and DFS traversal orders for the given tree highlights the fundamental differences in how these algorithms explore a graph. BFS explores all the neighbours at the current depth level before moving on to nodes at the next level, resulting in a level-by-level traversal of the tree that starts from the root (vertex 0) and expands outward. In contrast, DFS explores as far as possible along each branch before backtracking, leading to a traversal that delves deep